

Documentação

Técnica Oficial -

MusicReview

Projeto Full-Stack: Music Review

Desenvolvido para o Processo Trainee Comp Júnior
Samuel Aguiar Guimarães

1. Visão Geral do Projeto

O **Music Review** é uma aplicação Full-Stack desenvolvida para permitir que usuários descubram álbuns musicais utilizando a API oficial do Spotify e deixem avaliações (reviews) sobre eles. O projeto possui um ecossistema completo com controle de acesso, autenticação JWT, integração externa e banco de dados relacional, totalmente encapsulado via Docker.

2. Motivos das Escolhas Tecnológicas

Tecnologia	Justificativa da Escolha
Node.js + Express	Permite o uso de JavaScript em ambas as pontas (Front e Back), reduzindo a curva de aprendizado de uma nova linguagem. O Express é minimalista, focado e ideal para construir APIs RESTful de forma limpa.
PostgreSQL + Prisma ORM	O PostgreSQL garante consistência em dados relacionais (Usuários, Álbuns, Reviews). O Prisma ORM acelera o desenvolvimento eliminando a necessidade de escrever queries SQL puras repetitivas, além de tipar os dados automaticamente.
Docker + Compose	Garante que a aplicação rode perfeitamente em qualquer máquina ("works on my machine") isolando o banco de dados e os servidores Node.js. Facilita muito a avaliação do projeto.

3. Explicações das Lógicas Implementadas

Autenticação e Autorização (JWT)

- O fluxo de segurança utiliza JSON Web Tokens (JWT). Quando o usuário faz login, a senha é verificada (hash gerado com *bcryptjs*). Se for válida, o servidor assina um token contendo o ID e a Role (nível de acesso) do usuário. Em rotas sensíveis (como deletar contas ou álbuns), um *Middleware* intercepta a requisição, lê o token no cabeçalho (Bearer) e aprova ou bloqueia a ação (Retornando 401 ou 403).

Integração com a API do Spotify

- Para não precisar cadastrar álbuns manualmente, o backend atua como um *Proxy* para o Spotify. O servidor requisita um token temporário do Spotify e, em seguida, faz a busca pela query solicitada, retornando ao Front-end um JSON limpo e formatado.

Normalização de Dados do Spotify

- O Spotify trata álbuns, *singles* e faixas de forma distinta. O sistema aplica uma regra de normalização onde, independentemente do tipo original (album ou track), os dados são mapeados para uma estrutura comum (título, artista, capa) antes de serem persistidos no banco. Isso permite que o seu AlbumCard trate qualquer item musical com a mesma lógica de interface, independentemente da sua origem.

Hierarquia de Acessos e Proteção de Dados

- A regra de negócio para a exclusão de avaliações é baseada na propriedade do conteúdo. O servidor valida se o **user_id** extraído do token JWT é o mesmo **user_id** associado à review no banco de dados.

A role **ADMIN** atua como uma chave mestra. Usuários com esta permissão possuem autorização para listar todos os utilizadores do sistema (**GET /users**) e excluir registros de outros, permitindo a moderação da plataforma contra comentários ofensivos ou contas inativas.

Lógica de "Upsert" (Update ou Insert) em Avaliações

- Para evitar spam e manter a nota média do álbum precisa, o sistema não permite múltiplas avaliações do mesmo utilizador para o mesmo álbum. Ao submeter uma review, o servidor verifica a existência de uma avaliação prévia pelo par (user_id, album_id). Se encontrada, o sistema executa um update dos campos (nota e comentário). Caso contrário, executa um create. Isso assegura que cada utilizador tenha apenas uma voz ativa por obra, mantendo o histórico de críticas limpo.

4. Como rodar o projeto localmente

1. Preparando o Back-end

Clone o repositório

```
git clone https://github.com/SamuAguaHP/Music_Review.git
```

Acesse a pasta do back-end

```
cd Music_Review/backend
```

Instale as dependências

```
npm install
```

Crie um ficheiro .env na pasta backend (existe um .env.exemplo no GitHub) com as seguintes variáveis:

```
DATABASE_URL="postgresql://user:password@localhost:5433/music_review_db?schema=public"
```

```
JWT_SECRET="sua_chave_secreta"
```

Acesse: <https://developer.spotify.com> para conseguir essas ids

```
SPOTIFY_CLIENT_ID="seu_client_id_do_spotify"
```

```
SPOTIFY_CLIENT_SECRET="seu_client_secret_do_spotify"
```

Acesse: <https://mailtrap.io/home> para conseguir essas informações

```
MAIL_HOST="sandbox.smtp.mailtrap.io"
```

```
MAIL_PORT=2525
```

```
MAIL_USER="seu_user_mailtrap"
```

```
MAIL_PASS="seu_senha_mailtrap"
```

Suba o Banco de Dados e o Servidor:

Inicie os containers (PostgreSQL + Servidor Node) via Docker

`docker-compose up -d`

Sincronize as tabelas do Prisma com o banco

`docker-compose exec backend npx prisma migrate dev`

2. Preparando o Front-end

Em um novo terminal, acesse a pasta do front-end

`cd Music_Review/frontend`

Instale as dependências

`npm install`

Inicie o servidor Vite

`npm run dev`

Acesse a aplicação no navegador em: <http://localhost:5173>

5. Documentação dos Endpoints da API

Requer Token JWT no Header (Authorization: Bearer) quando indicado.

Método	Endpoint	Descrição	Acesso
POST	/auth/register	Cria novo usuário	Público
POST	/auth/login	Login e geração de JWT	Público
GET	/spotify/search?q={query}	Busca na API do Spotify	Autenticado*
GET	/albums	Lista todos os álbuns avaliados	Público
POST	/albums	Salvar álbum	Autenticado*

DELETE	/albums/:id	Exclusão de álbum	ADMIN*
GET	/reviews	Listar avaliação	Autenticado*
POST	/reviews	Criação de avaliação	Autenticado*
DELETE	/reviews/:id	Exclusão de avaliação	Autenticado*
GET	/users	Listar Usuários	ADMIN*
DELETE	/users/:id	Exclusão de usuário	Próprio ou ADMIN*